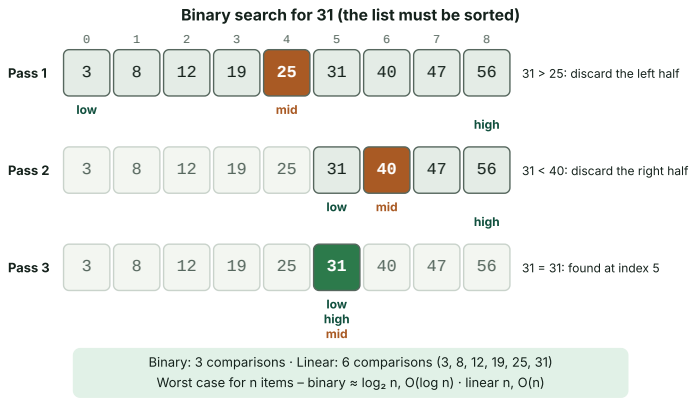
Searching algorithms

AP17 FACT FILE

Specification: students should be able to

- Describe how a linear search and a binary search work, using pseudocode, flowcharts and trace tables
- Implement linear search and binary search in C#
- Compare the efficiency of searching algorithms in terms of the number of comparisons in the best, average and worst cases
- Explain the conditions under which each search is appropriate

Key words: linear search, binary search, target, comparison, midpoint, sorted, integer division, divide and conquer, best case, average case, worst case, efficiency, time complexity, big-O notation, O(n), O(log n)



Binary search for 31 in a sorted array of nine items: each pass compares the middle item and discards half, finding it in 3 comparisons compared with 6 for a linear search

Searching: the big picture

Searching is one of the most common tasks a program carries out: checking a username at login, finding a product by its barcode or retrieving a patient's record in a hospital system. A search algorithm takes a collection of data and a target value and reports whether the target is present and, usually, its position. The two algorithms required are the linear search and the binary search. A linear search is simple and works on any list, sorted or not, but it may have to look at every item. A binary search is much faster on large lists because it discards half of the remaining data with each comparison, but it only works if the data is already sorted and can be accessed directly by position. Choosing between them is a classic trade-off between simplicity and speed. Efficiency is measured by counting comparisons, not by timing a program, because the count does not depend on the speed of a particular computer. The number of comparisons in the best, average and worst cases, and how that number grows as the list gets longer, is summarised using big-O notation.

Linear search

A linear, or sequential, search examines each item in turn from the start of the list, comparing it with the target. If they match, the search stops and returns the position; if the end of the list is reached without a match, the target is not present and a value such as -1 or a 'not found' message is returned. A well-written linear search stops as soon as the item is found, rather than carrying on to the end. Its strengths are simplicity and flexibility: it is easy to code correctly, works on unsorted data and can be used on data that can only be read in order, such as records in a serial file. Its weakness is speed. In the best case the target is the first item and only one comparison is needed, but in the worst case, when the target is last or missing, every item must be compared, so n comparisons are needed for a list of n items. On average about half the list is checked. The time grows in direct proportion to the size of the list, which is described as O(n), or linear time.

Binary search

A binary search works only on sorted data. Two pointers, low and high, mark the section of the list still being searched, starting at the first and last positions. The middle position is calculated as (low + high) DIV 2, using integer division so that it rounds down. The item at that position is compared with the target. If they are equal, the search ends. If the target is larger, it can only be in the upper half, so low is moved to mid + 1; if it is smaller, high is moved to mid − 1. The process repeats on the remaining half. If low becomes greater than high, no items are left to check and the target is not in the list. Because every comparison halves the search space, binary search is described as divide and conquer. Its best case is one comparison, when the target happens to be in the middle. Its worst case for n items is $\lfloor \log_2 n \rfloor + 1$ comparisons, so 1 000 items need at most 10 comparisons and a million need at most 20. This is O(log n), or logarithmic time.

Tracing a binary search

To trace a binary search, draw a table with columns for the pass number, low, high, mid, the item at mid and the action taken. Start with low at 0 and high at the last index. On each pass, calculate mid with integer division, look up the item, and decide whether it matches, whether low moves up to mid + 1, or whether high moves down to mid − 1. For the sorted list 2, 5, 9, 14, 21, 27, 33, 40, 46, 52, searching for 46 gives mid values of 4, 7 and 8, comparing with 21, 40 and 46, so the target is found at index 8 after three comparisons. Searching for 10 compares with 21, 5, 9 and 14; after the fourth comparison high becomes 2 while low is 3, so low is greater than high and the search reports that 10 is not present. Note that different textbooks round the midpoint differently or count positions from 1 rather than 0; in the exam, follow the algorithm you are given exactly and show your working so that the method can be credited.

Measuring efficiency

The efficiency of an algorithm is judged by how the number of steps it takes (time complexity) and the memory it uses (space complexity) grow as the size of the input increases, rather than by timing it on one computer, which would depend on the hardware. Analysts consider the best case, the average case and the worst case. Big-O notation expresses the order of growth, usually for the worst case, and ignores constant factors and smaller terms, because for large inputs only the dominant term matters; an algorithm that takes 3n + 2 steps is O(n). Common orders, from most to least efficient for large n, are O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n) and $O(n^2)$ quadratic. A logarithmic algorithm scales extremely well: doubling the size of a sorted list adds only one extra comparison to a binary search, whereas it doubles the worst-case work of a linear search. Both searches use only a few extra variables, so their space requirements are small and similar; the important difference between them is time.

Choosing a search

Binary search is not always the right choice. It requires the data to be sorted, and sorting has a cost: if a list is searched only once or twice, or if new items are added constantly in random order, the time spent sorting or keeping the list in order may outweigh the time saved by searching faster. It also needs direct access to any position in the list, so it suits arrays held in memory rather than data read one record at a time. For short lists, perhaps a few dozen items, the difference in speed is negligible and a linear search is simpler to write, test and maintain, with less risk of off-by-one errors in the low, high and mid calculations. Binary search is the better choice when a large data set is sorted, or can be kept sorted, and is searched frequently, such as a library catalogue, a dictionary or a product database searched thousands of times a day. In practice, large systems often use indexes or other structures, but the principle of reducing the number of comparisons is the same.

T Key terms from the mark schemes

Linear search
A search that checks each item in turn from the start until the target is found or the end of the list is reached.
`Credited: sequential search`

Binary search
A search of a sorted list that repeatedly compares the target with the middle item and discards the half that cannot contain it.

Divide and conquer
An approach that solves a problem by repeatedly reducing it to smaller sub-problems, as when binary search halves the list each time.

Time complexity
A measure of how the number of steps an algorithm takes grows as the size of its input grows.

Big-O notation
Notation that describes the order of growth of an algorithm's time or space, usually in the worst case, as input size increases, e.g. O(n).

Best case
The input that makes an algorithm do the least work, such as the target being the first item checked.

Logarithmic time
O(log n) growth, in which doubling the input size adds only a constant amount of extra work.
Credited: O(log n)

Worst case
The input that makes an algorithm do the most work, such as the target being last or absent in a linear search.

Integer division
Division that discards any remainder, so 13 DIV 2 = 6; used to calculate the midpoint in a binary search.
Credited: DIV

C C# example

Linear search in C#

```
static int LinearSearch(string[] list, string target)
{
    for (int i = 0; i < list.Length; i++)
    {
        if (list[i] == target) return i;
    }
    return -1;
}

string[] towns = { "Omagh", "Derry", "Belfast", "Enniskillen", "Newry" };
Console.WriteLine(LinearSearch(towns, "Enniskillen"));   // 3
Console.WriteLine(LinearSearch(towns, "Armagh"));        // -1
```

Exam focus

When asked to describe a binary search, examiners look for a sequence of clear points: the list must be sorted; find the middle item; compare it with the target; if it matches, stop; otherwise discard the half that cannot contain the target; repeat on the remaining half until the item is found or no items remain. Each point is typically worth one mark, so do not skip the stopping conditions. For a linear search, state that each item is checked in turn from the start until it is found or the end is reached. Trace questions may ask you to list the items compared, in order, or to complete a table of low, high and mid; show every pass and remember that integer division rounds down. Calculations of the maximum number of comparisons should show the halving or the power of two used. For 'compare' or 'evaluate' questions, make each point two-sided (linear needs no sorting, whereas binary does) and finish with a justified recommendation for the scenario. A common mistake is saying binary search is always better without mentioning the need for sorted data.

? Check yourself

1. State one condition that must be met before a binary search can be used. [1]
2. Describe how a linear search finds an item in a list. [3]
3. Describe how a binary search finds an item in a list. [5]

Answers: 1. The list/data must be sorted (into order) (1) 2. Start at the first item (1) / compare each item in turn with the target (1) / stop when the item is found or the end of the list is reached (target not present) (1) 3. List must be sorted (1) / find/compare the middle item with the target (1) / if equal, the item is found and the search stops (1) / if the target is less than the middle item, discard the upper half (or if greater, discard the lower half) (1) / repeat with the remaining half until found or no items remain (1)